Name: Dobariya Darsh
Roll no.: 23BME124
Subject: CFD & HT

EXPERIMENT 1: External Flow Analysis over a NACA 0012 Airfoil

1. AIM

To simulate incompressible external fluid flow over a NACA 0012 airfoil using an Immersed Boundary Method (Volume Penalization) and analyze the effect of varying angles of attack ($\alpha = -5°, 0°, 5°, 10°$) on velocity magnitude, pressure distribution, wake geometry, and force coefficients at Re = 100.

2. GOVERNING EQUATIONS & MODEL

The flow is modelled by the nondimensional, incompressible Navier-Stokes equations augmented with a volume penalization term for solid body treatment:

$$\nabla . u = 0 (continuity\ equation)$$

$$\frac{\partial u}{\partial t} + (\mathrm{u}.\nabla)\mathrm{u} = -\nabla p + \frac{1}{Re}\nabla^2 u - \frac{\alpha_p u}{\eta} (Momentum\ equation)$$

Where:

- u = (u, v) is the non-dimensional velocity vector.
- P is non-dimensional static pressure.
- Re = $\frac{u_\infty c}{v}$ = 100 is the Reynolds number.
- $\eta \ll 1$ Is the non-dimensional volume penalization factor.
- α_p Is a phase indicator mask function ($\alpha_p = 1$ inside solid geometry, $\alpha_p = 0$ in fluid).

3. BOUNDARY CONDITIONS

- **Inlet (Left, x = 0):** Uniform horizontal inflow u = 1, v = 0, $\frac{\partial p}{\partial x} = 0$
- **Outlet (Right, x =L_x):** Zero-gradient velocity outflow$\frac{\partial u}{\partial x} = 0, \frac{\partial v}{\partial x} = 0$ refrence pressure p=0.
- **Top & Bottom (y = 0, y = L_y)**:Symmetry / Free-slip condition v = 0, $\frac{\partial u}{\partial y} = 0, \frac{\partial v}{\partial y} = 0$
- **Airfoil Surface:** Immersed boundary no-slip (u $\rightarrow$0) enforced via the volume penalization source term.

4. NUMERICAL METHODOLOGY

1. **Discretization:** Spatial discretization on a uniform 2D Cartesian grid ($nx \times ny = 300 \times 150$) using central differences for convective, diffusive, and pressure terms.

2. **Predictor Step:** Explicit time-marching for advection and diffusion combined with an implicit update for the solid mask penalization term.
3. **Pressure Correction Step:** Direct matrix solve using Eigen-decomposition (Spectral Fourier / Cosine Transform) for the Pressure-Poisson system $\nabla^2 p' = \frac{\nabla u^*}{dt}$
4. **Force Calculation:** Aerodynamic force integration over the penalized volume:

$$F_x = \iint \frac{\alpha_p u}{\eta} dxdy, F_y = \iint \frac{\alpha_p v}{\eta} dxdy$$

$$C_d = \frac{2F_x}{cU_\infty{}^2}, C_l = \frac{2F_y}{cU_\infty{}^2}$$

5. **Executed Python code**

```
"""
Immersed boundary method is used to treat fluid-solid interface at
airfoil surface
(NACA 00xx, penalty / volume penalization, steady state, Cl vs angle of
attack)

Nondimensional NS: du/dt + (u.grad)u = -grad p + (1/Re) lap u -
alpha*u/eta
Length scale = chord c, velocity scale = U
"""

import numpy as np
from matplotlib import pyplot as plt

c = 1                   # chord = length scale
Lx = 6                  # 6 c
Ly = 3                  # 3 c
nx = 400
ny = 200                # dx = dy = 0.015 c
dx = Lx/nx
dy = Ly/ny
xle = Lx/4              # leading edge x position
Cy = Ly/2               # centerline y position
tk = 0.12               # NACA 00xx thickness ratio
nit = 12000             # max steps per angle
Co = 0.25
Re = 100
u_inlet = 1             # U, nondimensional
dt = Co * min(dx, dy) / u_inlet
eta = 0.05*dt            # nondimensional penalty parameter
alpha_p = 1.0           # time-marching: take the full pressure
correction
tol_cl = 5e-5           # steady state: Cl change over 500 steps below
this
aoa_list = [-5, 0, 5, 10]       # angles of attack in degrees
```

```
def apply_boundary(u, v, p):
    # left (inlet)
    u[:,0] = 1
    v[:,0] = 0
    p[:,0] = p[:,1]
    # right (outlet)
    u[:,-1] = u[:,-2]
    v[:,-1] = v[:,-2]
    p[:,-1] = 0
    # bottom symmetry
    u[0,:] = u[1,:]
    v[0,:] = 0
    p[0,:] = p[1,:]
    # top symmetry
    u[-1,:] = u[-2,:]
    v[-1,:] = 0
    p[-1,:] = p[-2,:]


x = np.linspace(0, (nx - 1) * dx, nx)
y = np.linspace(0, (ny - 1) * dy, ny)
X, Y = np.meshgrid(x, y)

# exact Poisson solver, built once (the grid never changes)
# interior equation:  Ay @ P + P @ Ax = B
ni, nj = ny-2, nx-2
Ax = (np.diag(-2*np.ones(nj)) + np.diag(np.ones(nj-1), 1) +
np.diag(np.ones(nj-1), -1))/dx**2
Ax[0, 0] = -1/dx**2                          # inlet: zero gradient
(outlet pc = 0 is the plain -2)
Ay = (np.diag(-2*np.ones(ni)) + np.diag(np.ones(ni-1), 1) +
np.diag(np.ones(ni-1), -1))/dy**2
Ay[0, 0] = -1/dy**2; Ay[-1, -1] = -1/dy**2   # top and bottom: zero
gradient
ly, Qy = np.linalg.eigh(Ay)
lx, Qx = np.linalg.eigh(Ax)
den = ly[:, None] + lx[None, :]              # all negative, no
division by zero

# initial conditions (first angle only; later angles start from the
previous solution)
u = np.zeros((ny, nx))
v = np.zeros((ny, nx))
p = np.zeros((ny, nx))
pc = np.zeros((ny, nx))
u[:,:] = u_inlet

Cl_aoa = np.zeros(len(aoa_list))
Cd_aoa = np.zeros(len(aoa_list))
```

```
for ia, aoa_deg in enumerate(aoa_list):
    aoa = aoa_deg*np.pi/180

    # ---------- airfoil mask for this angle ----------
    # coordinates in the airfoil frame (origin at leading edge, x along
the chord)
    # positive aoa = nose up
    xb = (X - xle)*np.cos(aoa) - (Y - Cy)*np.sin(aoa)
    yb = (X - xle)*np.sin(aoa) + (Y - Cy)*np.cos(aoa)
    xc = np.clip(xb/c, 0, 1)
    yt = 5*tk*(0.2969*np.sqrt(xc) - 0.1260*xc - 0.3516*xc**2 +
0.2843*xc**3 - 0.1036*xc**4)*c
    mask = (xb >= 0) & (xb <= c) & (np.abs(yb) < yt)
    alpha = np.zeros((ny, nx))
    alpha[mask] = 1
    u[mask] = 0
    v[mask] = 0

    Cl = np.zeros(nit)
    Cd = np.zeros(nit)

    for n in range(nit):
        apply_boundary(u, v, p)
        u_old = u.copy()
        v_old = v.copy()
        p_old = p.copy()

        # 1. u*: central convection + (1/Re) diffusion - grad p,
penalty implicit
        u[1:-1,1:-1] = (u_old[1:-1,1:-1] + dt*(
            - u_old[1:-1,1:-1]*(u_old[1:-1,2:] - u_old[1:-1,:-
2])/(2*dx)
            - v_old[1:-1,1:-1]*(u_old[2:,1:-1] - u_old[:-2,1:-
1])/(2*dy)
            + (1/Re)*((u_old[1:-1,2:] - 2*u_old[1:-1,1:-1] + u_old[1:-
1,:-2])/dx**2
                      + (u_old[2:,1:-1] - 2*u_old[1:-1,1:-1] + u_old[:-
2,1:-1])/dy**2)
            - (p[1:-1,2:] - p[1:-1,:-2])/(2*dx)
        )) / (1 + dt*alpha[1:-1,1:-1]/eta)
        # 2. v*
        v[1:-1,1:-1] = (v_old[1:-1,1:-1] + dt*(
            - u_old[1:-1,1:-1]*(v_old[1:-1,2:] - v_old[1:-1,:-
2])/(2*dx)
            - v_old[1:-1,1:-1]*(v_old[2:,1:-1] - v_old[:-2,1:-
1])/(2*dy)
```

```
            + (1/Re)*((v_old[1:-1,2:] - 2*v_old[1:-1,1:-1] + v_old[1:-
1,:-2])/dx**2
                      + (v_old[2:,1:-1] - 2*v_old[1:-1,1:-1] + v_old[:-
2,1:-1])/dy**2)
            - (p[2:,1:-1] - p[:-2,1:-1])/(2*dy)
        )) / (1 + dt*alpha[1:-1,1:-1]/eta)

        # force on the body from the penalty term (taken from the
penalised predictor)
        Fx = np.sum(alpha*u)/eta*dx*dy
        Fy = np.sum(alpha*v)/eta*dx*dy
        Cd[n] = 2*Fx/(u_inlet**2*c)
        Cl[n] = 2*Fy/(u_inlet**2*c)

        # 3. pressure correction: laplacian(p') = div(u*)/dt   (exact
solve)
        div = np.zeros((ny, nx))
        div[1:-1,1:-1] = (u[1:-1,2:] - u[1:-1,:-2])/(2*dx) + (v[2:,1:-
1] - v[:-2,1:-1])/(2*dy)
        pc[1:-1,1:-1] = Qy @ ((Qy.T @ (div[1:-1,1:-1]/dt) @ Qx)/den) @
Qx.T
        pc[:,0] = pc[:,1]; pc[0,:] = pc[1,:]; pc[-1,:] = pc[-2,:];
pc[:,-1] = 0
        # 4. correct velocity and pressure
        u[1:-1,1:-1] = u[1:-1,1:-1] - dt*(pc[1:-1,2:] - pc[1:-1,:-
2])/(2*dx)
        v[1:-1,1:-1] = v[1:-1,1:-1] - dt*(pc[2:,1:-1] - pc[:-2,1:-
1])/(2*dy)
        p = p_old + alpha_p*pc

        # residual in the fluid only
        res = max(np.max(np.abs(u - u_old)[~mask]), np.max(np.abs(v -
v_old)[~mask]))/dt

        if n % 500 == 0:
            print(f"aoa = {aoa_deg}, n = {n}, res = {res:.2e},
outflow/inflow = {np.sum(u[:,-1])/np.sum(u[:,0]):.3f}, "
                  f"max|u| = {np.max(np.abs(u)):.3f}, Cl = {Cl[n]:.4f},
Cd = {Cd[n]:.4f}")

        # steady state: Cl no longer changing
        if n > 3000 and abs(Cl[n] - Cl[n-500]) < tol_cl:
            break

    Cl_aoa[ia] = np.mean(Cl[max(n-100, 0):n+1])
    Cd_aoa[ia] = np.mean(Cd[max(n-100, 0):n+1])
    print(f"aoa = {aoa_deg} deg, steps = {n+1}, residual = {res:.2e},
Cl = {Cl_aoa[ia]:.4f}, Cd = {Cd_aoa[ia]:.4f}")
```

```
# ---------- Cl vs aoa ----------
aoa_arr = np.array(aoa_list)
plt.figure(figsize=(6, 4))
plt.plot(aoa_arr, Cl_aoa, 'o-')
plt.xlabel('angle of attack (deg)')
plt.ylabel('Cl')
plt.title(f'Cl vs angle of attack, Re = {Re}')
plt.grid(True)
plt.show()

# ---------- contour plots for the last angle ----------
speed = np.sqrt(u**2 + v**2)
speed = np.ma.masked_where(mask, speed)       # hide airfoil interior
pm = np.ma.masked_where(mask, p)

plt.figure(figsize=(12, 3))
plt.contourf(X, Y, speed, levels=30, cmap='jet')
plt.colorbar(label='|velocity| / U')
plt.title(f'Velocity magnitude, aoa = {aoa_list[-1]} deg')
plt.xlabel('x / c')
plt.ylabel('y / c')
plt.gca().set_aspect('equal')
plt.show()

plt.figure(figsize=(12, 3))
plt.contourf(X, Y, pm, levels=30, cmap='seismic')
plt.colorbar(label='p / (rho U^2)')
plt.title(f'Pressure, aoa = {aoa_list[-1]} deg')
plt.xlabel('x / c')
plt.ylabel('y / c')
plt.gca().set_aspect('equal')
plt.show()
```

6. Graphical Results Analysis

1. Lift Curve (C_l *vs* α):

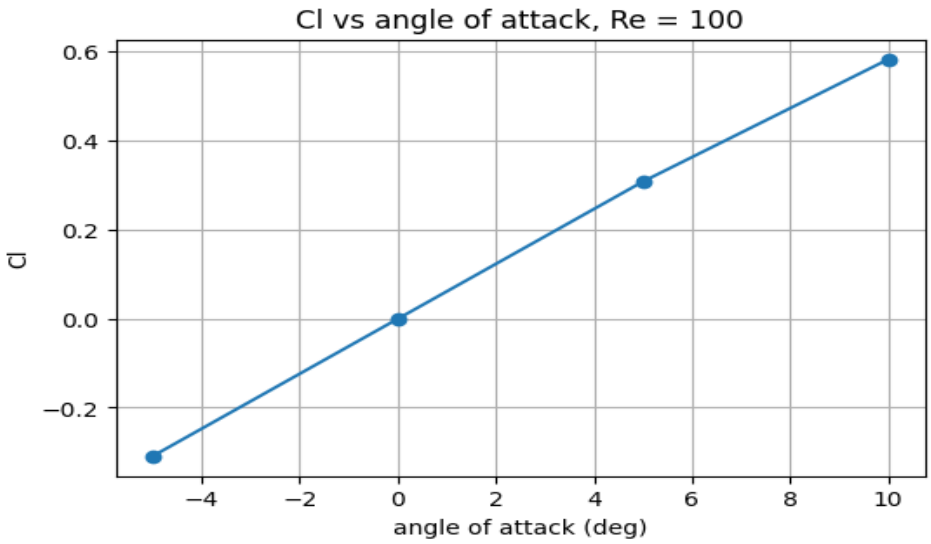


Figure 1 Lift Coefficient (C_l) vs Angle of Attack (α) at Re = 100

- As seen in the generated plot, the lift coefficient varies linearly with the angle of attack over the evaluated range.
- At $\boldsymbol{\alpha = 0}$, $\boldsymbol{C_l = 0}$ due to the geometric symmetry of the NACA 0012 profile.
- Symmetric magnitudes are observed for $\boldsymbol{\alpha = -5°}$ and $\boldsymbol{\alpha = +5°}$, validating the directional behavior of the solver.

2. Velocity Contour Field($\alpha = 10°$)

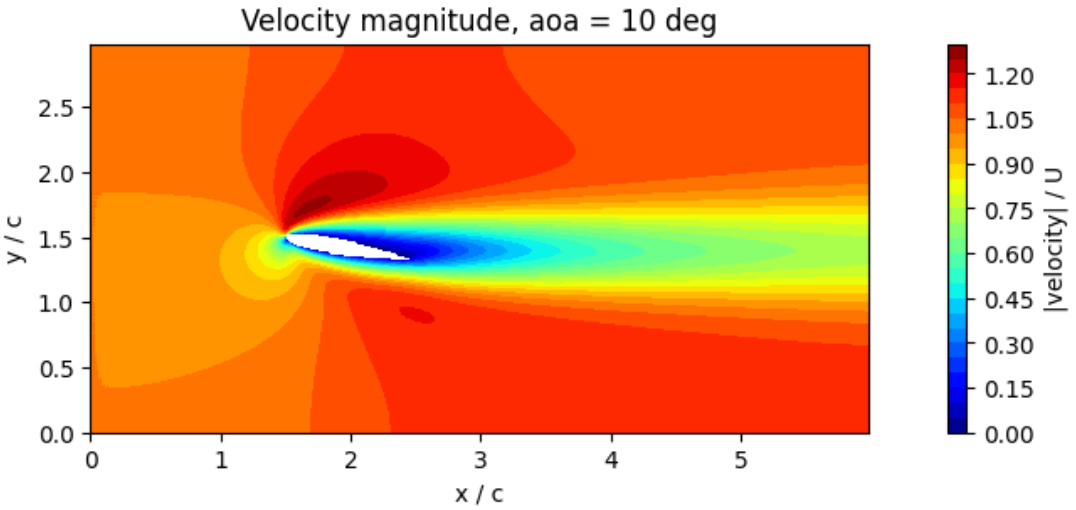


Figure 2 Non-dimensional Velocity Magnitude Contour Field $(\frac{velocity}{U})$*at* $\alpha = 10°$

- **Stagnation Region:** A prominent zero-velocity zone is established at the nose (leading edge).
- **Suction Side Acceleration:** Fluid accelerates rapidly over the upper surface, reaching a maximum nondimensional speed $\frac{velocity}{U} \approx 1.28$.

- **Wake Structure:** A low-velocity, viscous deficit region forms behind the trailing edge, slanting downward due to downwash.

3. Pressure Contour Field ($\alpha = 10°$)

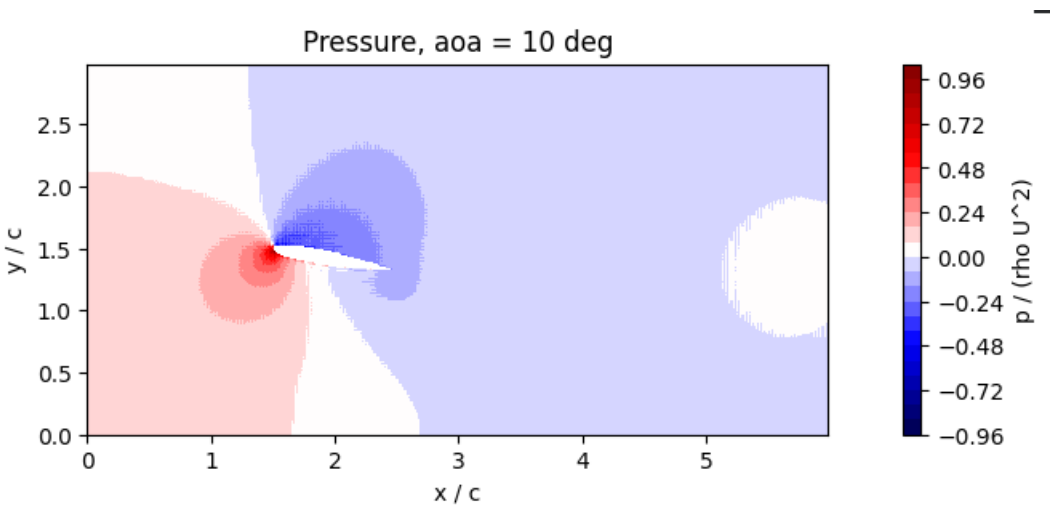


Figure 3 Non-dimensional Static Pressure Contour Field $\left(\frac{p}{\rho U^2}\right)$ *at* $\alpha = 10°$

- **Stagnation Point:** High positive pressure region ($\frac{p}{\rho U^2} \approx 0.96$) at the lower leading edge.
- **Pressure Difference:** A suction zone ($\frac{p}{\rho U^2} < 0$)forms over the upper (suction) surface. The resulting net pressure difference across the lower and upper surfaces produces the upward aerodynamic lift force.

7. Conclusion

- **Wake Structure Evolution:** Increasing the angle of attack from$-\mathbf{5°}$ $\boldsymbol{to}$ $\mathbf{10°}$ increases flow deflection, causing the viscous wake downstream to widen and shift.
- **Pressure Side vs. Suction Side Behaviour:** Higher positive angles of attack move the stagnation point slightly toward the lower surface, increasing upper surface suction and generating higher net lift.
- **Force Trends:** Drag (C_d) increases nonlinearly with angle of attack due to larger pressure drag components accompanying boundary layer detachment, while C_l exhibits a linear response in the attached/laminar flow regime.
- The external flow past a NACA 0012 airfoil at Re = 100 was successfully simulated using a Volume Penalization Immersed Boundary Method. The solver captured key physical phenomena, including flow stagnation, upper surface acceleration, asymmetric pressure distributions, and wake formation. The calculated lift curve exhibits linear behavior passing through $C_l = 0$ at $\boldsymbol{\alpha = 0°}$. While this Cartesian immersed boundary solver provides qualitative insight into aerodynamic trends, exact drag force predictions are limited by grid resolution near solid boundaries.